COMP 316 PROJECT

Fake News Detection Using NLP Techniques and Machine Learning Models

(MAY 2026)

Group Members:
Arya Maharaj (224131564)
Moyantha Ayair (224017564)
Jerrell Chetty (224007944)

NLP PROBLEM

The widespread distribution of false information through online platforms has made fake news detection a formidable problem in today's society. Fake news articles often mimic legitimate reporting in structure and tone, while conveying misleading or fabricated information, making large-scale manual verification impractical.

Natural Language Processing (NLP) offers automated methods for analyzing text and detecting characteristic patterns that separate fake news from real news.

This project focuses on building and evaluating machine learning models that can classify news articles based on their textual content. The objective is not only to achieve high classification performance, but also to investigate which NLP techniques and machine learning models are most suitable for this task.

SOLUTION APPROACH

Pipeline:

Our solution follows the standard NLP pipeline:
Data → Preprocessing → Vectorization →Model Training → Evaluation →Comparison

Each vectorization technique and model was implemented as an interchangeable component, meaning the dataset, preprocessing steps, and evaluation metrics remained identical across all experiments, with only the vectorizer or model being swapped.
This modular design ensures that all experiments are conducted under consistent conditions, enabling fair and systematic comparison across configurations.

Dataset description:

The dataset used in this study was compiled from two sources, comprising approximately 23,500 fake news articles and 21,500 real news articles, resulting in a total of roughly 45,000 samples. It was sourced from Kaggle and consists of two CSV files - Fake.csv and True.csv - which were merged and labelled accordingly.

The near-equal class distribution means the dataset is approximately balanced, which is important because it ensures that classification metrics such as accuracy are meaningful and not skewed toward a dominant class.

Each article consists of full textual content paired with a corresponding binary label indicating whether it is fake (0) or real (1).
The dataset is characterized by long-form text, with each sample containing multiple sentences and a large number of words. Its vocabulary is diverse, encompassing both common words and topic-specific terminology. Furthermore, while fake and real articles may appear structurally similar, they tend to differ at the vocabulary level in terms of specific word choice and repetition patterns.

These characteristics directly shaped our selection of NLP techniques, which are detailed in the "NLP techniques" section.

Preprocessing:

Before model training, preprocessing was applied to clean and standardize the text. This involved lowercasing the text to ensure consistency- so that words like "Trump" and "trump" are treated as the same token- as well as removing punctuation and other noise to eliminate characters that carry no semantic value for classification. Stopwords such as "the" and "is" were also removed to eliminate common words that do not contribute meaningfully to the task. The text was then tokenized into individual words, and lemmatization was applied to reduce each word to its base form, improving overall dataset consistency. Together, these steps reduce noise and help the model focus on meaningful linguistic patterns rather than irrelevant variations in the text.

NLP Techniques:

Several NLP techniques were selected based on the characteristics of the dataset and the requirements of the classification task.

Count Vectorization was used as a baseline technique. It works by counting how many times each word appears in an article and using those counts as features for the model. Its simplicity makes it computationally efficient and suitable for probabilistic models such as Naive Bayes.
However, the downside is that it does not account for the importance of words across the dataset - it gives equal weight to every word. This means that frequently occurring but non-discriminative words may still influence the model, even if they do not contribute meaningful information for classification.

TF-IDF (Term Frequency - Inverse Document Frequency) was selected as a more advanced alternative. It improves on Count Vectorization by not just counting words, but by assigning weights to words based on how unique they are across the corpus. Words that appear frequently in a specific article but rarely in others, get a higher score, meaning it is treated as more significant. This allows it to reduce the influence of common words.
Formally, the TF-IDF score for a term t in document d is calculated as:
$TF\text{-}IDF(t, d) = TF(t, d) \times \log(N / df(t))$
where $TF(t, d)$ is the frequency of term t in document d, N is the total number of documents in the corpus, and $df(t)$ is the number of documents containing term t.
This technique is particularly useful for this dataset because words that are specific to fake or real news will be weighted more heavily than words that appear everywhere. Thus it helps reduce noise from common words and it highlights distinguishing terms, hence it is better suited for long documents.

N-grams (Unigrams and Bigrams) were incorporated to capture contextual information. While unigrams consider individual words, bigrams capture pairs of consecutive words, allowing the model to learn short phrases. This is important because fake news often relies on repeated multi-word expressions and stylistic patterns that are not captured when words are treated independently. Therefore, bigrams were expected to improve performance by introducing limited contextual awareness.

Word2vec (embeddings) was explored as an embedding-based technique. It represents words as dense vectors where words with similar meanings are positioned close together. While this is a powerful technique, this approach focuses on semantic similarity between words rather than identifying specific discriminative vocabulary. For a binary classification task such as fake news

detection, where specific word usage is more important than semantic relationships, Word2Vec was not expected to outperform TF-IDF. It was therefore included for experimental comparison rather than as a primary method.

Given the dataset's long text structure and reliance on vocabulary patterns, TF-IDF and bigrams were expected to perform best, as they emphasise word importance and contextual patterns.

Machine learning models:

A range of machine learning models was selected to represent different learning paradigms, enabling a comprehensive comparison of how probabilistic, linear, and non-linear approaches perform on the same task.

Naive Bayes is a probabilistic classifier that assumes independence between features. It is widely used in text classification due to its efficiency and ability to handle high-dimensional data. It performs particularly well with count-based representations and is expected to behave as a strong baseline model. However, its independence assumption limits its ability to capture relationships between words. It was not applied to Word2Vec representations, as it requires non-negative feature values, whereas Word2Vec produces continuous vectors that may contain negative values.

Logistic Regression is a linear classifier that models the probability of class membership. It performs well with high-dimensional sparse data, making it highly suitable for TF-IDF representations. It is known for producing stable and generalisable results in text classification tasks.

Support Vector Machine (SVM) is another linear model that aims to find an optimal decision boundary between classes. It is effective in high-dimensional spaces and is known for strong generalisation performance, particularly in text classification problems. It was selected to represent a second linear approach for direct comparison with Logistic Regression.

Linear Support Vector Classification (LinearSVC) is a computationally efficient variant of SVM that is specifically optimized for linear decision boundaries and large-scale datasets. Unlike kernel-based SVM, LinearSVC scales well to high-dimensional sparse representations, making it well-suited to Count and TF-IDF vectorization. It was included as an additional model to complement the standard SVM and provide a direct comparison of linear classification approaches.

Decision Tree is a non-linear model that makes decisions based on feature splits. It is capable of capturing complex relationships in the data and is easy to interpret. However, it is prone to overfitting, especially on large and high-dimensional datasets such as text data. It was included specifically to evaluate whether a non-linear approach offers any advantage over linear models for this task.

Neural models were intentionally excluded due to their computational complexity and the project's focus on comparing interpretable classical techniques under controlled experimental conditions.

Additional models considered but not implemented include **Random Forest**, which performs poorly on high-dimensional sparse text vectors due to its reliance on random feature subsampling, making it less suited to vocabulary-based classification. **BERT and other transformer-based**

models were also considered but excluded due to their high computational requirements, need for GPU resources, and the project's focus on comparing classical interpretable techniques under controlled conditions.

Experimental methodology:

All experiments were conducted under identical conditions to ensure that observed differences in performance could be attributed solely to the choice of vectorization technique or model, rather than any external factor.
The dataset was split into 80% training and 20% testing, with the same split applied consistently across all experiments. Preprocessing was performed once and reused across all configurations. Only the vectorization technique or model was varied per experiment, with all other components of the pipeline remaining fixed. This controlled setup enables fair, valid, and reproducible comparison across all configurations tested.

Each model was evaluated using a consistent set of metrics. **Accuracy** measured the overall proportion of correctly classified articles. **Precision** captured how many of the articles predicted as fake were actually fake, while **recall** assessed how many of the genuinely fake articles the model correctly identified. The **F1-score**, being the harmonic mean of precision and recall, provided a single balanced measure of performance. **Confusion matrices** were also used to examine error patterns directly, showing the breakdown of true positives, true negatives, false positives, and false negatives. F1-score was treated as the primary metric since it balances precision and recall - both matter here, as missing a fake article and incorrectly flagging a real one carry different but significant consequences.

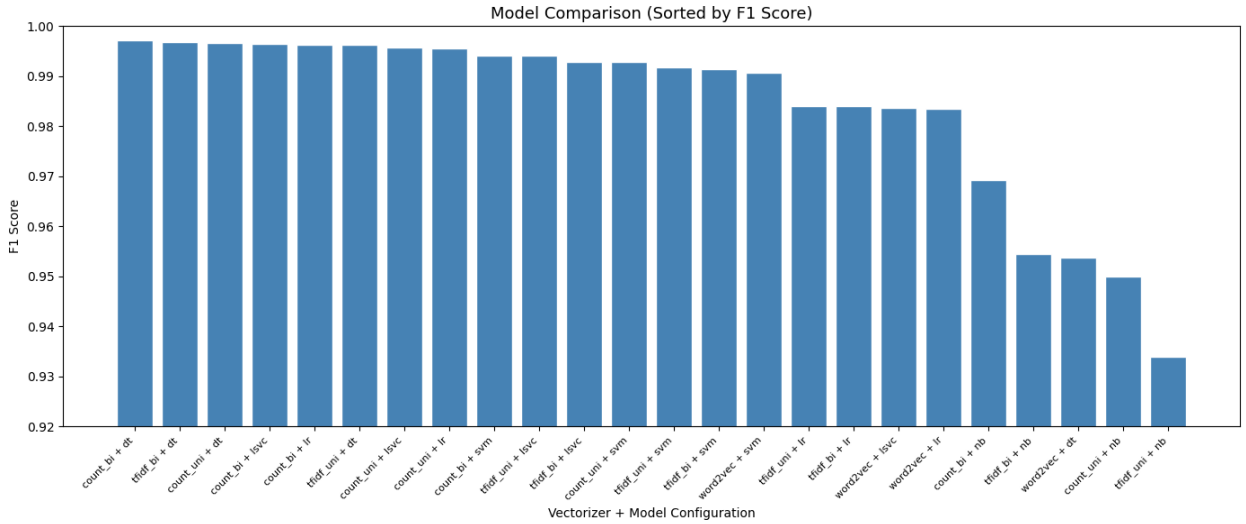
Additional metrics such as the Area Under the ROC Curve (AUC-ROC) and Matthews Correlation Coefficient (MCC) were considered. AUC-ROC measures discriminative ability across all classification thresholds and is particularly valuable for imbalanced datasets; however, given that our dataset is approximately balanced (50/50), accuracy and F1 are already reliable and meaningful, making AUC-ROC a less critical addition. MCC provides a single correlation-based measure that accounts for all four quadrants of the confusion matrix and is robust under class imbalance. As the dataset is balanced and confusion matrices were reported, the selected metrics provide sufficient interpretive coverage for this experimental context.

RESULTS

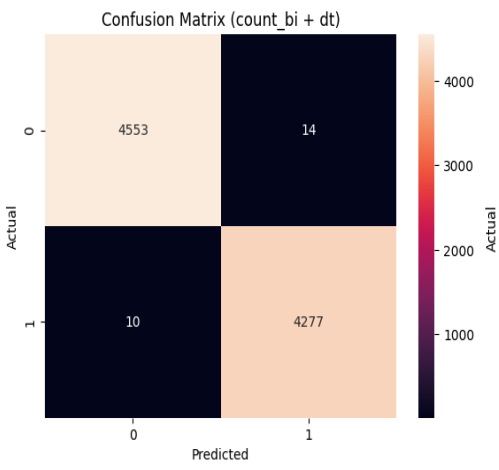
Table 1 below presents accuracy, precision, recall, and F1-score for all 24 experimental configurations tested. It is sorted by descending F1 -score.

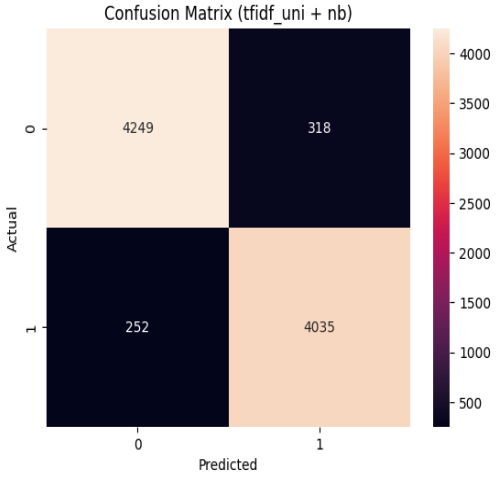
Vectorizer	Model	Accuracy	Precision	Recall	F1
count_bi	dt	0.9973	0.9967	0.9977	0.9972
tfidf_bi	dt	0.9970	0.9963	0.9974	0.9969
count_uni	dt	0.9967	0.9956	0.9977	0.9966
count_bi	lsvc	0.9965	0.9951	0.9977	0.9964
count_bi	lr	0.9964	0.9960	0.9965	0.9963
tfidf_uni	dt	0.9963	0.9947	0.9977	0.9962
count_uni	lsvc	0.9959	0.9944	0.9972	0.9958
count_uni	lr	0.9957	0.9953	0.9958	0.9956
count_bi	svm	0.9942	0.9937	0.9944	0.9941
tfidf_uni	lsvc	0.9942	0.9935	0.9946	0.9941
tfidf_bi	lsvc	0.9931	0.9900	0.9958	0.9929
count_uni	svm	0.9930	0.9935	0.9921	0.9928
tfidf_uni	svm	0.9921	0.9909	0.9928	0.9918
tfidf_bi	svm	0.9916	0.9886	0.9942	0.9914
word2vec	svm	0.9910	0.9870	0.9944	0.9907
tfidf_uni	lr	0.9846	0.9835	0.9848	0.9841
tfidf_bi	lr	0.9845	0.9814	0.9867	0.9841
word2vec	lsvc	0.9841	0.9781	0.9893	0.9836
word2vec	lr	0.9838	0.9796	0.9872	0.9834
count_bi	nb	0.9701	0.9625	0.9762	0.9693
tfidf_bi	nb	0.9555	0.9449	0.9643	0.9545
word2vec	dt	0.9556	0.9605	0.9473	0.9538
count_uni	nb	0.9511	0.9432	0.9566	0.9499
tfidf_uni	nb	0.9356	0.9269	0.9412	0.9340

Figure 1 below presents a comparison of all 24 experimental configurations sorted by F1 score, allowing for direct visual comparison of model and vectorizer performance.



The highest F1 score overall was achieved by Count Vectorization with bigrams and Decision Tree (F1 = 0.9972), followed closely by TF-IDF with bigrams and Decision Tree (F1 = 0.9969). Naïve Bayes consistently produced the lowest F1 scores across all vectorizers, while Logistic Regression, SVM, and LinearSVC showed the most stable performance across different configurations. LinearSVC achieved competitive F1 scores ranging from 0.9836 to 0.9964 across all five vectorizers.





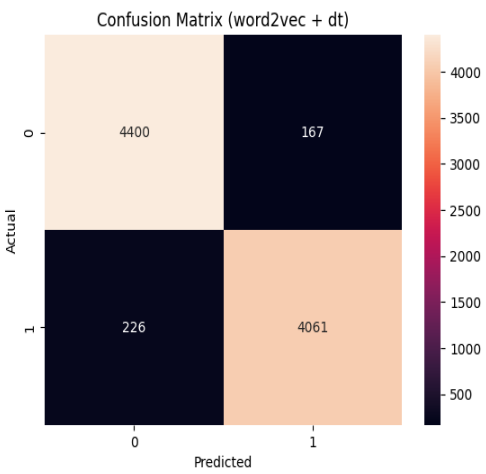


Figure 2 presents the confusion matrix for the **best performing** configuration, Count Vectorization with bigrams and Decision Tree (F1 = 0.9972). The matrix shows very few misclassifications, with only a small number of fake articles incorrectly classified as real and vice versa, confirming the high F1 score achieved by this configuration.

Figure 3 presents the confusion matrix for the **weakest performing** configuration, TF-IDF with unigrams and Naïve Bayes (F1 = 0.9340). Compared to the best performer, a notably higher number of misclassifications are observed in both directions, reflecting the limitations of Naïve Bayes's independence assumption when applied to TF-IDF weighted features.

Figure 4 presents the confusion matrix for Word2Vec with Decision Tree (F1 = 0.9538), which produced the **most unexpected** result. Although the Decision Tree performed strongly across all bag-of-words configurations, its performance dropped significantly with Word2Vec, suggesting that dense semantic vectors interact poorly with decision tree splitting criteria compared to sparse frequency-based representations.

DATA ANALYSIS

The results reveal clear performance trends across both vectorization techniques and machine learning models.

Overall, Decision Tree achieved the highest F1 scores across most configurations, with the best result obtained using Count Vectorization with bigrams (F1=0.9972). However, this consistently high performance suggests potential overfitting, which is a known limitation of decision trees when applied to high-dimensional data.

Logistic Regression, SVM, and LinearSVC demonstrated strong and consistent performance across all vectorization techniques, with F1 scores typically above 0.98. LinearSVC in particular performed comparably to Logistic Regression across all vectorizers, confirming its suitability for high-dimensional text classification. These models showed greater stability compared to Decision Trees across different configurations.

In terms of vectorization techniques, bigram representations consistently outperformed unigram equivalents, particularly for Count Vectorization. This confirms that incorporating short phrase patterns improves classification performance.

Interestingly, TF-IDF did not significantly outperform Count Vectorization despite expectations, suggesting that the dataset is sufficiently separable using raw frequency patterns alone. Word2Vec achieved competitive performance with Logistic Regression and SVM, but showed notably lower performance with Decision Tree, suggesting inconsistent behaviour across model types compared to bag-of-words techniques.

DISCUSSIONS OF THE RESULTS

The experimental results largely supported the expectations outlined during technique selection, though some findings were unexpected. The observed results can be explained by considering both the nature of the dataset and the properties of the techniques used.

The strong performance of Decision Trees can be attributed to their ability to capture complex patterns, non-linear patterns in high-dimensional data. However, their consistently high accuracy likely reflects overfitting to the training data rather than genuine generalisation. This means that while they performed well on the test split, their reliability on completely unseen real-world data may be limited. For practical applications, this makes them a less trustworthy choice despite their high accuracy.

Logistic Regression and SVM performed consistently well because text data represented through TF-IDF and Count Vectorization is inherently high-dimensional and sparse - exactly the type of data these linear models are designed for. Their resistance to overfitting makes them more reliable for generalisation, which is more valuable in a real-world fake news detection scenario than marginal accuracy gains. LinearSVC performed comparably to Logistic Regression across all vectorizers, which is expected given both are linear classifiers optimised for high-dimensional sparse data, further confirming that linear models generalise well on this task.

The improvement observed with bigrams confirms that contextual information plays an important role in this task. Individual words alone do not always capture the patterns that distinguish fake

from real news - certain phrases and word combinations carry meaning that single tokens cannot represent. This explains why bigram configurations consistently outperformed their unigram equivalents.

Although TF-IDF was expected to significantly outperform Count Vectorization, the results suggest that the dataset is relatively easy to separate, meaning simple frequency-based features are already highly informative. This highlights that more complex feature weighting does not always lead to significant improvements, and raises the possibility of dataset bias- the models may be learning source-specific stylistic patterns rather than genuinely understanding what makes content misleading.

This observation is corroborated by reviewing published work on the same Kaggle dataset, where comparable configurations consistently report high F1 scores (often exceeding 0.98), with Logistic Regression and TF-IDF frequently cited as the most reliable combination. Acknowledging this limitation demonstrates that our analysis extends beyond surface-level performance reporting to a critical evaluation of the dataset itself, situating our findings within the broader literature.

The slightly lower performance of Word2Vec can be explained by the nature of the task. Fake news detection relies on identifying specific discriminative vocabulary rather than understanding semantic relationships between words. Word2Vec excels at the latter but is less suited to tasks where the presence of particular words matters more than their meaning relative to other words.

CONCLUSION

This project demonstrates that traditional NLP techniques combined with classical machine learning models are highly effective for fake news classification.

While Decision Trees achieved the highest F1- score, their tendency to overfit suggests that Logistic Regression and SVM provide more reliable and generalisable performance.

The best overall performing combination was Count Vectorization with bigrams and Decision Tree, achieving the highest F1-score. However, for practical applications, TF-IDF combined with Logistic Regression, LinearSVC, or SVM offers a better balance between performance and generalisation.

The results highlight the importance of feature representation, with bigrams improving performance by capturing contextual information. Additionally, the findings show that simpler techniques such as Count Vectorization can perform competitively with more advanced methods like TF-IDF, depending on the dataset.

Overall, the project confirms that model and technique selection must be guided by the characteristics of the dataset, rather than assuming more complex methods will always produce better results. These findings demonstrate that effective NLP solutions depend not only on model performance, but on understanding how linguistic features interact with machine learning techniques.